

Ultimate Production Readiness Cheat Sheet for Event-Sourced Ingestion & Risk Prediction ML Workflows

Use case: Event-sourced ingestion systems, current-state projections, risk scoring, fraud/ATO models, compliance scoring, security telemetry, abuse detection, anomaly detection, & any ML workflow built from immutable event history.

Table of Contents

- **1. EXECUTIVE RELEASE DECISION**

- A. [*Required decision rules*](#)
- B. [*Production-ready definition*](#)
- C. [*Stop-ship examples*](#)
- D. [*Runbook TODO*](#)
- E. [*Notes*](#)

- **2. SEVERITY CLASSIFICATION**

- A. [*Severity table*](#)
- B. [*PM decision mapping*](#)
- C. [*Stop-ship examples*](#)
- D. [*Runbook TODO*](#)
- E. [*Notes*](#)

- **3. CORE ARCHITECTURE CHECKLIST**

- A. [*Required controls*](#)
- B. [*Required questions*](#)
- C. [*Validation evidence*](#)
- D. [*Stop-ship examples*](#)
- E. [*Runbook TODO*](#)
- F. [*Notes*](#)

- **4. EVENT-SOURCING CORRECTNESS**

- A. [*Required controls*](#)
- B. [*Required questions*](#)
- C. [*Required tests*](#)
- D. [*Stop-ship examples*](#)
- E. [*Runbook TODO*](#)
- F. [*Notes*](#)

- **5. IDEMPOTENCY, DEDUPLICATION & AT-LEAST-ONCE DELIVERY**

- A. [*Required controls*](#)
- B. [*Required questions*](#)
- C. [*Required tests*](#)
- D. [*Stop-ship examples*](#)
- E. [*Runbook TODO*](#)
- F. [*Notes*](#)

- **6. ORDERING, PARTITIONING & CONCURRENCY**

- A. [Required controls](#)
- B. [Required questions](#)
- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

- **7. HTTP RATE-LIMIT HEADER HANDLING & DYNAMIC THROTTLING**

- A. [Headers to inspect](#)
- B. [Required controls](#)
- C. [Dynamic throttle algorithm](#)
- D. [Required tests](#)
- E. [Stop-ship examples](#)
- F. [Runbook TODO](#)
- G. [Notes](#)

- **8. TIMEOUTS, RETRIES, BACKOFF & JITTER**

- A. [Required timeout hierarchy](#)
- B. [Required retry policy](#)
- C. [Jitter modes](#)
- D. [Required tests](#)
- E. [Stop-ship examples](#)
- F. [Runbook TODO](#)
- G. [Notes](#)

- **9. ERROR HANDLING & DATA VALIDATION**

- A. [Error categories](#)
- B. [Required handling](#)
- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

- **10. SECURITY & SECRET HANDLING**

- A. [Required controls](#)
- B. [Required tests](#)
- C. [Stop-ship examples](#)
- D. [Runbook TODO](#)
- E. [Notes](#)

- **11. PRIVACY, COMPLIANCE & AUDITABILITY**

- A. [Required controls](#)
- B. [Required audit metadata](#)
- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

- **12. ML FEATURE ENGINEERING CORRECTNESS**

- A. [Required controls](#)
- B. [Required questions](#)

- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

• 13. ML MODEL VALIDATION & ACCEPTANCE

- A. [Required validation](#)
- B. [Recommended metrics](#)
- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

• 14. MODEL SERVING & ONLINE SCORING

- A. [Required controls](#)
- B. [Required prediction metadata](#)
- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

• 15. SCALABILITY & PERFORMANCE

- A. [Required controls](#)
- B. [Performance metrics](#)
- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

• 16. HORIZONTAL SCALING, LAMBDA & SANDBOXING

- A. [Required controls](#)
- B. [Sandbox metrics](#)
- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

• 17. OBSERVABILITY, LOGGING, METRICS & ALERTING

- A. [Required identifiers](#)
- B. [Required metrics](#)
- C. [Severity alerts](#)
- D. [Required tests](#)
- E. [Stop-ship examples](#)
- F. [Runbook TODO](#)
- G. [Notes](#)

• 18. TESTING MATRIX

- A. [Unit tests](#)
- B. [Validation tests](#)
- C. [Regression tests](#)
- D. [Integration tests](#)

- E. [Contract tests](#)
- F. [Fault-injection tests](#)
- G. [Security tests](#)
- H. [ML tests](#)
- I. [Performance tests](#)
- J. [Notes](#)

- **19. CI/CD QUALITY GATES**

- A. [Required pre-merge gates](#)
- B. [Required coverage](#)
- C. [Required release gates](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

- **20. DEPLOYMENT, CANARY, FEATURE FLAGS & ROLLBACK**

- A. [Required deployment controls](#)
- B. [Canary stages](#)
- C. [Canary metrics](#)
- D. [Rollback triggers](#)
- E. [Rollback requirements](#)
- F. [Stop-ship examples](#)
- G. [Runbook TODO](#)
- H. [Notes](#)

- **21. SYSTEM PACKAGE, DEPENDENCY, VENDOR & LICENSE GOVERNANCE**

- A. [Required controls](#)
- B. [Vendor maintenance criteria](#)
- C. [License checks](#)
- D. [Required metrics](#)
- E. [Required gates](#)
- F. [Stop-ship examples](#)
- G. [Runbook TODO](#)
- H. [Notes](#)

- **22. CONFIGURATION, POLICY & MIGRATION**

- A. [Required controls](#)
- B. [Required questions](#)
- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

- **23. COST, CAPACITY & OPERATIONAL EFFICIENCY**

- A. [Required cost controls](#)
- B. [Required questions](#)
- C. [Required tests](#)
- D. [Stop-ship examples](#)
- E. [Runbook TODO](#)
- F. [Notes](#)

- **24. DOCUMENTATION & RUNBOOKS**

- A. [Required documentation](#)
- B. [Required runbook actions](#)
- C. [Stop-ship examples](#)
- D. [Final acceptance checklist](#)
- E. [Notes](#)

1. EXECUTIVE RELEASE DECISION

1.A. Required decision rules

- a. **Do not ship** if any Sev-1 issue exists.
- b. **Do not promote to production** if any Sev-2 issue lacks:
 - i. mitigation
 - ii. owner
 - iii. rollout guard
 - iv. rollback path
 - v. expiry date
- c. **Do not use ML predictions for enforcement** unless the model has passed:
 - i. held-out validation
 - ii. calibration checks
 - iii. drift checks
 - iv. threshold review
 - v. shadow evaluation
 - vi. canary evaluation
- d. **Do not ingest externally influenced data** without:
 - i. validation
 - ii. bounded resource controls
 - iii. observability
 - iv. DLQ/quarantine
 - v. replay safety
- e. **Do not deploy any artifact** without:
 - i. SBOM
 - ii. vulnerability scan
 - iii. license scan
 - iv. secret scan
 - v. version pinning
 - vi. reproducible build metadata
- f. **Do not release** if rollback, kill switch, or emergency disablement is untested.

1.B. Production-ready definition

A service is production-ready only when it is:

- a. Correct under duplicate delivery.
- b. Correct under retries.
- c. Correct under partial failures.
- d. Correct under out-of-order delivery where ordering is not globally guaranteed.
- e. Safe under load.
- f. Safe under malicious or malformed input.
- g. Observable during incidents.
- h. Auditable after incidents.
- i. Rollback/canary capable.
- j. Secure by default.
- k. Compliant with data, license, vendor, & productization requirements.
- l. Tested across unit, validation, regression, integration, fault-injection, security, ML, & deployment paths.

1.C. Stop-ship examples

- a. Credential material enters immutable logs.
- b. Consumer can acknowledge failed work.
- c. Duplicate events corrupt projections.
- d. Model is promoted based on training accuracy.
- e. No rollback path exists.
- f. No SBOM or license scan exists.
- g. Critical CVE exists in a production artifact.
- h. No event replay/rebuild validation exists.

1.D. Runbook TODO

- a. Define release approval owner.
- b. Define Sev-1/Sev-2 escalation owner.
- c. Define model enforcement owner.
- d. Define rollback commander.
- e. Define customer-impact communication path.
- f. Define compliance/legal approval path for dependency, dataset, model, & vendor usage.

1.E. Notes

- a. For AI-generated code, default to stricter review until tests prove correctness.
- b. Event ingestion correctness, credential safety, & model validity are non-negotiable gates.

2. SEVERITY CLASSIFICATION

2.A. Severity table

Sl. #	Severity	Meaning	Examples	Required action
1	Sev-1	Stop-ship risk	Credential leak, event loss, projection corruption, invalid enforcement model, license violation	Block release
2	Sev-2	Must fix before production	Duplicate mishandling, missing DLQ, missing observability, high model false positives	Fix or formally mitigate
3	Sev-2.5	High priority	Cost spike, hot partition, vendor risk, missing dashboard	Fix before broad rollout
4	Sev-3	Cleanup	Naming, minor docs, style, low-risk refactor	Track normally

2.B. PM decision mapping

Sl. #	Issue type	PM decision
1	Sev-1	Stop-ship
2	Sev-2 without mitigation	Do not launch
3	Sev-2 with owner, expiry, guardrail, & rollback	Limited canary only
4	Sev-2.5	Canary allowed with monitoring
5	Sev-3	Does not block launch

2.C. Stop-ship examples

- a. Source-of-truth data can be corrupted.
- b. State projections can drift silently.
- c. Failed events can be acknowledged.
- d. Secrets, tokens, credentials, or sensitive PII can leak.
- e. Model predictions can enforce incorrect decisions.
- f. Release includes incompatible license or unapproved commercial-use asset.

2.D. Runbook TODO

- a. Create severity rubric.
- b. Map severities to alerting channels.
- c. Map severities to PM release decisions.
- d. Define exception process, waiver owner, expiry, & remediation issue.

2.E. Notes

- a. Sev-2.5 is useful for risks that may not block QA but should block broad rollout.

3. CORE ARCHITECTURE CHECKLIST

3.A. Required controls

- a. Immutable event log is the source of truth.
- b. Event schemas are versioned.
- c. Events are append-only & never edited in place.
- d. Events contain only safe, durable, auditable data.
- e. Secrets, plaintext credentials, raw tokens, private keys, session IDs, & unnecessary sensitive PII are never written to immutable logs.
- f. Projections are disposable & rebuildable from event history.
- g. Projection rebuilds are deterministic for domain state.
- h. Consumers are idempotent.
- i. Duplicate delivery is expected & tested.
- j. Ordering is enforced or validated per aggregate/account/entity/partition.
- k. State writes are atomic & version-checked.
- l. Consumers ack only after durable success.
- m. Failures are retried, dead-lettered, or quarantined based on explicit policy.
- n. ML feature generation is point-in-time correct.
- o. Online scoring uses the same feature schema as offline training.
- p. Models are versioned, monitored, canary-enabled, & rollback-capable.

3.B. Required questions

- a. What is the source of truth?
- b. Which state is derived?
- c. Can projections be rebuilt from scratch?
- d. What is the aggregate key?
- e. What is the partition key?
- f. What is the idempotency key?
- g. How are duplicates handled?
- h. How are gaps handled?
- i. How are unknown events handled?
- j. How are schema versions handled?
- k. How is the model version tied to predictions?

- I. How is rollback performed?

3.C. Validation evidence

Sl. #	Area	Required evidence
1	Event schema	Versioned schema fixtures
2	Replay	Deterministic replay tests
3	Idempotency	Duplicate redelivery tests
4	Projection writes	Optimistic concurrency tests
5	ML scoring	Offline/online feature parity tests
6	Deployment	Canary & rollback plan
7	Supply chain	SBOM, CVE, license reports

3.D. Stop-ship examples

- a. Plaintext secret or credential material in events, state, logs, features, or model artifacts.
- b. No idempotency.
- c. No event version validation.
- d. No replay test.
- e. Acking failed processing.
- f. Silent payload parse failures.
- g. Unknown event type advancing projection version.
- h. Model trained & evaluated on the same data.
- i. Accuracy-only metric for rare-event prediction.
- j. No deployment rollback path.

3.E. Runbook TODO

- a. Document event append process.
- b. Document projection rebuild process.
- c. Document model deployment process.
- d. Document rollback process.
- e. Document DLQ replay process.

3.F. Notes

- a. Source-of-truth durability & derived-state rebuildability should be explicit in design docs.

4. EVENT-SOURCING CORRECTNESS

4.A. Required controls

- a. Each event has a stable event ID.
- b. Each event has a monotonic aggregate version.
- c. Each event has an explicit aggregate/account/entity ID.
- d. Each event has an explicit partition/shard/space key.
- e. Event time is distinct from processing time.
- f. Event schema version is explicit.
- g. Producer identity is recorded.
- h. Append is atomic.

- i. Expected current version is checked during append.
- j. Projection update is atomic.
- k. Projection update is guarded by expected previous version.
- l. Replay is deterministic.
- m. Duplicate, stale, & gap versions are handled explicitly.
- n. Unknown event types are rejected, quarantined, or handled through compatibility rules.
- o. Schema migrations are tested against historical events.
- p. Operational metadata is separated from deterministic domain state.

4.B. Required questions

- a. Does `current_version + 1 == event.version` for normal application?
- b. What happens when `event.version <= current_version` ?
- c. What happens when `event.version > current_version + 1` ?
- d. What happens when the same `event_id` has the same payload?
- e. What happens when the same `event_id` has a different payload?
- f. What happens when the same aggregate version has a different event ID?
- g. Does an unknown event type advance projection state?
- h. Does an invalid event payload mutate projection state?
- i. Does ack occur only after durable processing or explicit DLQ/quarantine write?

4.C. Required tests

- a. Replay from genesis to expected state.
- b. Replay same stream twice produces identical domain state.
- c. Duplicate event does not change state twice.
- d. Stale version is ignored or quarantined.
- e. Version gap is delayed, retried, or quarantined.
- f. Same version/different event ID is rejected.
- g. Same event ID/different payload is rejected.
- h. Unknown event type does not advance projection.
- i. Malformed event does not mutate projection.
- j. Concurrent same-aggregate events preserve correctness.
- k. Different aggregates can process concurrently.
- l. Projection rebuild matches live projection.
- m. Historical schema fixtures remain replayable.

4.D. Stop-ship examples

- a. Projection version is set directly from event without expected-version check.
- b. State save uses last-writer-wins.
- c. Consumer processes the same aggregate concurrently without serialization or optimistic locking.
- d. Event replay produces different domain state on each run.
- e. Unknown event type advances version.
- f. Invalid payload produces zero-value mutation.
- g. Failed event is acknowledged.

4.E. Runbook TODO

- a. Add projection rebuild command.
- b. Add projection checksum command.
- c. Add event gap inspection command.
- d. Add event quarantine inspection command.
- e. Add safe replay procedure.

4.F. Notes

- a. Deterministic domain state should not include `now()` values unless they are event-derived or separated as operational metadata.

5. IDEMPOTENCY, DEDUPLICATION & AT-LEAST-ONCE DELIVERY

5.A. Required controls

- a. Stable idempotency key per operation.
- b. Stable event ID per appended event.
- c. Processed-event tracking or strict aggregate version enforcement.
- d. Atomic write of projection update & processed-event marker.
- e. Duplicate-safe counters.
- f. Duplicate-safe list/set updates.
- g. Retry-safe external calls.
- h. Retry-safe writes.
- i. Retry-safe model feature updates.
- j. Retry-safe alert generation.
- k. Retry-safe side effects.
- l. No double-counting from redelivery.
- m. No duplicate password reset, notification, payment, lockout, or enforcement side effect.

5.B. Required questions

- a. What happens when the same event is delivered twice?
- b. What happens when the same event is delivered 100 times?
- c. What happens when `SaveState` succeeds but `ack` fails?
- d. What happens when `ack` succeeds but logging fails?
- e. What happens when `append` succeeds but client times out?
- f. What happens when downstream dependency partially succeeds?
- g. Are side effects idempotent?
- h. Are external writes protected by idempotency keys?
- i. Is duplicate detection durable across restarts?

5.C. Required tests

- a. Duplicate delivery storm.
- b. Ack failure followed by redelivery.
- c. Save success plus ack failure.
- d. Save failure plus no ack.
- e. Retry after timeout.
- f. Duplicate non-GET operation with idempotency key.
- g. Counter increments once.
- h. Set/list update remains bounded & deduped.
- i. Notification/enforcement side effect fires once.

5.D. Stop-ship examples

- a. Duplicate event increments counter twice.
- b. Duplicate event appends duplicate list element.
- c. Retry creates duplicate enforcement action.
- d. Processed-event marker is not durable.
- e. Save succeeds but ack failure causes unsafe duplicate mutation.

5.E. Runbook TODO

- a. Add duplicate-event inspection query.
- b. Add idempotency-key lookup.
- c. Add side-effect reconciliation procedure.
- d. Add duplicate storm mitigation procedure.

5.F. Notes

- a. At-least-once delivery makes idempotency mandatory, not optional.

6. ORDERING, PARTITIONING & CONCURRENCY

6.A. Required controls

- a. Explicit partition key.
- b. Explicit aggregate/entity key.
- c. Per-aggregate serialization or optimistic concurrency.
- d. Bounded worker pools.
- e. Bounded queues.
- f. Semaphores for concurrency control.
- g. Global concurrency limits.
- h. Per-tenant limits.
- i. Per-endpoint limits.
- j. Per-aggregate or hot-key controls.
- k. Backpressure when downstream services slow down.
- l. Graceful shutdown that drains or safely releases work.
- m. Cancellation propagation through async tasks/promises/goroutines/futures.
- n. No unbounded goroutine/task creation.

6.B. Required questions

- a. Is ordering guaranteed by the broker, partitioner, consumer, or database?
- b. What happens during consumer rebalance?
- c. What happens when two workers process the same aggregate?
- d. Can a hot key starve other work?
- e. Are work queues bounded?
- f. Are semaphores used around expensive calls?
- g. Are promises/futures awaited & cancelled safely?
- h. Are tasks cleaned up on timeout?
- i. Are file descriptors & sockets released?

6.C. Required tests

- a. Same aggregate concurrent delivery.
- b. Out-of-order event delivery.
- c. Partition rebalance.
- d. Worker crash mid-event.
- e. Graceful shutdown.
- f. Hot key load.
- g. Large burst load.
- h. Bounded queue overflow.
- i. Semaphore limit enforcement.
- j. Cancellation cleanup.
- k. Race detector / data-race tests.
- l. Lease/heartbeat loss.

6.D. Stop-ship examples

- a. One goroutine/task per event with no bound.
- b. Same aggregate processed concurrently.
- c. No backpressure.
- d. No graceful shutdown.
- e. No race test for concurrent state updates.

6.E. Runbook TODO

- a. Add hot-partition detection guide.
- b. Add worker-drain procedure.
- c. Add queue throttle procedure.
- d. Add rebalance incident procedure.

6.F. Notes

- a. Ordering should be treated as a correctness boundary, not only a performance concern.

7. HTTP RATE-LIMIT HEADER HANDLING & DYNAMIC THROTTLING

7.A. Headers to inspect

Sl. #	Header	Purpose
1	Retry-After	Required wait before retry
2	X-RateLimit-Limit	Provider-specific request limit
3	X-RateLimit-Remaining	Remaining budget
4	X-RateLimit-Reset	Reset timestamp or delay
5	X-RateLimit-Reset-After	Reset delay
6	X-RateLimit-Policy	Provider policy metadata
7	RateLimit-Limit	Standardized request limit
8	RateLimit-Remaining	Standardized remaining budget
9	RateLimit-Reset	Standardized reset
10	RateLimit-Policy	Standardized policy
11	X-Quota-Limit	Quota limit
12	X-Quota-Remaining	Remaining quota
13	X-Quota-Reset	Quota reset
14	X-Request-Cost	Request cost
15	X-Usage	Usage metadata
16	X-Used	Used budget
17	X-Account-Usage	Account-level usage
18	X-API-Usage	API-level usage
19	X-App-Usage	App-level usage

Sl. #	Header	Purpose
20	X-Page-Usage	Page-level usage
21	X-Client-Trace-Id	Provider-side trace ID
22	Provider-specific cost/quota headers	Vendor-specific throttle signal

7.B. Required controls

- a. Read rate-limit & retry headers from every HTTP response where applicable.
- b. Dynamically throttle clients based on server-provided limits.
- c. Use local adaptive throttling when headers are missing or inconsistent.
- d. Prefer provider-specific semantics when documented.
- e. Never ignore `Retry-After` on 429 or 503.
- f. Never retry all clients simultaneously after the same delay without jitter.
- g. Never exceed configured tenant, provider, endpoint, or token budgets.
- h. Track rate-limit state per provider, endpoint, tenant, token, region, & method where relevant.
- i. Emit throttle metrics for limit, remaining, reset, retry-after wait, effective concurrency, & throttled requests.

7.C. Dynamic throttle algorithm

- a. Maintain token bucket or leaky bucket per dependency/endpoint/tenant.
- b. Update bucket capacity from `RateLimit-Limit` or `X-RateLimit-Limit`.
- c. Update remaining tokens from `RateLimit-Remaining` or `X-RateLimit-Remaining`.
- d. Update next refill from `RateLimit-Reset` or `X-RateLimit-Reset`.
- e. If remaining is low, reduce concurrency proactively.
- f. If remaining is zero, pause until reset plus jitter.
- g. If headers disagree, choose the safest lower-throughput interpretation.
- h. If headers are missing, fall back to local adaptive throttling.
- i. If headers are malformed, log structured warning & use local throttling.
- j. Apply AIMD:
 - i. Additive increase on sustained success & healthy latency.
 - ii. Multiplicative decrease on 429, 503, timeout, p99 latency regression, retry-budget burn, or error spike.

7.D. Required tests

- a. 429 with `Retry-After` seconds.
- b. 429 with `Retry-After` HTTP date.
- c. 503 with `Retry-After`.
- d. `X-RateLimit-Remaining` reaches zero.
- e. `RateLimit-Reset` malformed.
- f. Missing rate-limit headers.
- g. Conflicting rate-limit headers.
- h. Provider lowers limit mid-run.
- i. Provider raises limit mid-run.
- j. Retry deadline exceeded.
- k. Jitter prevents synchronized retry storm.
- l. Per-tenant throttle isolation.
- m. Per-endpoint throttle isolation.
- n. Adaptive throttle reduces concurrency on p99 latency regression.
- o. Adaptive throttle recovers after healthy period.

7.E. Stop-ship examples

- a. Ignoring `Retry-After`.
- b. Retrying immediately after 429.
- c. Retrying all workers at the same timestamp.

- d. No max retry delay.
- e. No retry deadline.
- f. No per-tenant quota isolation.
- g. No metrics for throttling.
- h. No backpressure during throttle pause.
- i. Header parsing failure causes unlimited retries.

7.F. Runbook TODO

- a. Document provider-specific rate-limit semantics.
- b. Document throttle override procedure.
- c. Document emergency traffic shed procedure.
- d. Document vendor quota escalation path.
- e. Document replay procedure after throttle backlog.

7.G. Notes

- a. `Retry-After` should override normal retry timing unless unsafe or malformed.
- b. Jitter is required to avoid retry storms.

8. TIMEOUTS, RETRIES, BACKOFF & JITTER

8.A. Required timeout hierarchy

- a. DNS timeout.
- b. TCP connect timeout.
- c. TLS handshake timeout.
- d. Request write timeout.
- e. First-byte timeout.
- f. Read timeout.
- g. Idle connection timeout.
- h. Total request timeout.
- i. Per-event processing timeout.
- j. Per-job timeout.
- k. End-to-end workflow deadline.
- l. Model scoring timeout.
- m. Batch training timeout.
- n. Lambda/container execution timeout.

8.B. Required retry policy

- a. Retry only retryable failures.
- b. Do not retry validation failures indefinitely.
- c. Do not retry authentication/authorization failures blindly.
- d. Use capped exponential backoff.
- e. Use jitter.
- f. Enforce retry budgets.
- g. Enforce max attempts.
- h. Enforce total deadline.
- i. Emit retry metrics.
- j. Attach idempotency keys to retryable non-GET operations.
- k. Honor server rate-limit headers.
- l. Use circuit breakers to avoid hammering unhealthy dependencies.
- m. Use hedged requests only for safe idempotent reads where duplicate work is acceptable.
- n. Cancel slower hedged requests after first success.

8.C. Jitter modes

Sl. #	Mode	Use case
1	Full jitter	Default for large distributed systems
2	Equal jitter	Smoother retry spread
3	Decorrelated jitter	Avoids lockstep retries
4	Provider delay plus spread	Use when provider specifies exact delay

8.D. Required tests

- a. Timeout does not leak resources.
- b. Retryable 500 retries.
- c. Non-retryable 400 does not retry.
- d. 401/403 route to credential/auth failure.
- e. 409 conflict uses optimistic concurrency path.
- f. 413 payload too large rejects.
- g. 415 unsupported media type rejects.
- h. 422 validation failure rejects.
- i. 429 throttles.
- j. 500/502/503/504 retry with backoff.
- k. Retry budget exhaustion sends to DLQ.
- l. Circuit breaker opens & half-opens.
- m. Hedged request cancels loser.
- n. Job deadline stops retries.

8.E. Stop-ship examples

- a. No timeout.
- b. Infinite retry.
- c. Retrying non-idempotent writes without idempotency key.
- d. No jitter.
- e. No retry budget.
- f. No circuit breaker around failing dependency.
- g. Deadline ignored.

8.F. Runbook TODO

- a. Document retry policy by error class.
- b. Document dependency-specific timeout defaults.
- c. Document circuit-breaker reset procedure.
- d. Document retry storm mitigation.

8.G. Notes

- a. Retries are a reliability tool only when bounded, observable, & idempotent.

9. ERROR HANDLING & DATA VALIDATION

9.A. Error categories

Sl. #	Category	Examples
1	Network	DNS failure, connection refused, connection reset, TLS/cert failure, socket timeout
2	IO	Read timeout, write timeout, partial read, broken pipe
3	HTTP client	400, 401, 403, 404, 408, 409, 413, 415, 422, 425, 429
4	HTTP server	500, 501, 502, 503, 504, 507, 508
5	Data format	Malformed JSON, NDJSON, HTML, XML, CSV, multipart
6	Payload	Unknown content type, oversized payload, truncated body, decompression failure, checksum mismatch
7	Schema	Schema mismatch, missing field, wrong type, invalid timestamp, invalid enum, invalid ID
8	Storage	Timeout, optimistic lock conflict, transaction conflict, partial write, stale read
9	Broker	Ack failure, redelivery, visibility timeout, poison message
10	ML	Model unavailable, feature unavailable, schema mismatch, prediction timeout, NaN/Inf

9.B. Required handling

- a. All errors are classified.
- b. All errors have retryability metadata.
- c. All errors have severity metadata.
- d. All errors produce structured logs.
- e. Sensitive values are redacted.
- f. Correlation IDs are preserved.
- g. Retryable errors retry with budget.
- h. Non-retryable errors go to DLQ/quarantine or user-safe failure path.
- i. Unknown errors fail closed.
- j. Validation errors never mutate durable state.
- k. Fatal data integrity errors page.

9.C. Required tests

- a. One test per error category.
- b. Validation errors do not ack as success unless DLQ write succeeds.
- c. Retryable errors preserve idempotency.
- d. Error logs contain correlation IDs.
- e. Error logs do not contain secrets or PII.
- f. Unknown errors fail closed.
- g. Panic/exception recovery path preserves event metadata.

9.D. Stop-ship examples

- a. JSON parse error ignored.
- b. Malformed payload mutates state.
- c. Unknown content type accepted.
- d. Unknown error treated as success.
- e. Sensitive error response logged.
- f. Poison event loops forever without DLQ.

9.E. Runbook TODO

- a. Document error classification.
- b. Document DLQ inspection.
- c. Document poison-event handling.
- d. Document validation failure triage.
- e. Document replay-after-fix process.

9.F. Notes

- a. All input from event streams, HTTP APIs, queues, files, & model features should be treated as untrusted.

10. SECURITY & SECRET HANDLING

10.A. Required controls

- a. No plaintext passwords.
- b. No raw reset tokens.
- c. No raw session tokens.
- d. No API keys in code, logs, events, configs, or traces.
- e. No private keys in repo or containers.
- f. No secrets in immutable logs.
- g. Passwords hashed with approved password hashing algorithm.
- h. Tokens stored hashed or encrypted with scoped key.
- i. Constant-time comparison for token/secret checks where applicable.
- j. TLS enforced.
- k. mTLS where needed.
- l. Scoped service credentials.
- m. Least privilege access.
- n. Secret rotation.
- o. Secret scanning in CI.
- p. PII redaction.
- q. Secure audit logging.
- r. Dependency vulnerability scanning.
- s. Container image scanning.
- t. SAST.
- u. DAST where applicable.
- v. IaC scanning.
- w. Runtime security policy.
- x. Network egress controls.

10.B. Required tests

- a. Secret scan.
- b. PII log redaction.
- c. Password hashing test.
- d. Token expiry/single-use test.
- e. Constant-time comparison policy test.
- f. Authn/authz tests.
- g. TLS config test.
- h. Container hardening test.
- i. Dependency vulnerability test.
- j. SAST findings gate.
- k. Egress restriction test.
- l. Negative tests for injection/SSRF/path traversal where applicable.

10.C. Stop-ship examples

- a. Plaintext password in state.
- b. Secret in event payload.
- c. Token logged.
- d. Raw IP/user data logged without policy.
- e. Secrets committed to repo.
- f. Critical CVE in production artifact.
- g. Unauthenticated admin endpoint.
- h. Unscoped cloud credentials.
- i. Dependency with known exploit & no mitigation.

10.D. Runbook TODO

- a. Secret rotation guide.
- b. Credential leak response.
- c. PII leak response.
- d. Container compromise response.
- e. CVE emergency patch procedure.

10.E. Notes

- a. Immutable event logs amplify the severity of secret leaks because historical data cannot simply be edited away.

11. PRIVACY, COMPLIANCE & AUDITABILITY

11.A. Required controls

- a. Data classification.
- b. Data minimization.
- c. PII inventory.
- d. Sensitive field allowlist/blocklist.
- e. Immutable event payload policy.
- f. Retention policy.
- g. Deletion/minimization strategy where legally required.
- h. Access control for event logs.
- i. Access control for feature stores.
- j. Access control for model training data.
- k. Audit trail for administrative actions.
- l. Audit trail for model training.
- m. Audit trail for model deployment.
- n. Audit trail for threshold changes.
- o. Audit trail for policy changes.
- p. Third-party data provenance.
- q. Customer data segregation.
- r. Tenant isolation.
- s. Encryption at rest.
- t. Encryption in transit.
- u. Key rotation.
- v. Regional data handling where applicable.

11.B. Required audit metadata

Sl. #	Metadata	Required for
1	event_id	Event traceability
2	aggregate_id	Projection traceability
3	tenant_id	Tenant isolation
4	partition_id	Ordering analysis
5	event_type	Schema/application logic
6	event_version	Replay correctness
7	schema_version	Compatibility
8	producer	Source attribution
9	actor	User/system actor
10	occurred_at	Domain time
11	ingested_at	Processing time
12	trace_id	Distributed trace
13	request_id	Request trace
14	model_version	ML prediction audit
15	feature_schema_version	ML feature audit
16	decision_policy_version	Enforcement audit
17	deployment_version	Release traceability

11.C. Required tests

- a. Sensitive fields blocked from immutable events.
- b. PII redaction in logs.
- c. Access-control tests for event logs.
- d. Access-control tests for feature stores.
- e. Retention policy tests where applicable.
- f. Audit event completeness.
- g. Model decision traceability.
- h. Tenant isolation tests.

11.D. Stop-ship examples

- a. Immutable event contains raw secrets.
- b. No access control for event log.
- c. No audit trail for risk decision.
- d. No model version tied to prediction.
- e. No retention policy for logs/features.
- f. No provenance for training dataset.
- g. No license rights for dataset/model use.

11.E. Runbook TODO

- a. Audit reconstruction guide.
- b. Data access review process.
- c. Retention exception procedure.
- d. Tenant data isolation incident guide.

11.F. Notes

- a. For prediction systems, auditability must cover data, features, model version, threshold, & final decision.

12. ML FEATURE ENGINEERING CORRECTNESS

12.A. Required controls

- a. Point-in-time feature correctness.
- b. No future leakage.
- c. Explicit feature schema.
- d. Feature versioning.
- e. Offline/online feature parity.
- f. Stable feature ordering.
- g. Missing-value policy.
- h. Cold-start policy.
- i. Drift detection.
- j. Feature freshness monitoring.
- k. Feature store or deterministic feature build.
- l. Bounded memory feature generation.
- m. Incremental feature updates where needed.
- n. Reproducible training data snapshots.

12.B. Required questions

- a. Does each training example have a score time?
- b. Are features computed only from data available before score time?
- c. Is the label window after the feature window?
- d. Are features available online at scoring time?
- e. Is feature order explicit?
- f. Is feature schema stored with model artifact?
- g. Are nulls/missing values handled?
- h. Are categorical values encoded consistently?
- i. Are rare categories handled?
- j. Are feature distributions monitored?
- k. Are high-cardinality fields handled safely?
- l. Are PII fields minimized or transformed?
- m. Are features stable under duplicate events?
- n. Are features stable under late events?

12.C. Required tests

- a. Future event excluded from features.
- b. Label window starts after feature window.
- c. Offline/online feature parity.
- d. Feature schema mismatch rejected.
- e. Missing feature fallback.
- f. Cold-start account/entity scoring.
- g. Duplicate event does not double-count feature.
- h. Late event handling.
- i. High-cardinality category handling.
- j. NaN/Inf rejection.
- k. Feature drift alert.
- l. Feature freshness alert.

12.D. Stop-ship examples

- a. Full account history used to score a past event.
- b. Training labels leak into features.
- c. Feature vector uses implicit dict/map ordering.
- d. Online scorer crashes for unseen account/entity.
- e. Missing values crash scoring.
- f. Feature schema not stored with model.
- g. Raw PII logged during feature generation.

12.E. Runbook TODO

- a. Feature drift investigation guide.
- b. Feature backfill guide.
- c. Feature schema migration guide.
- d. Offline/online mismatch triage guide.

12.F. Notes

- a. Point-in-time correctness is required for any event-history ML workflow.

13. ML MODEL VALIDATION & ACCEPTANCE

13.A. Required validation

- a. Train/validation/test split.
- b. Time-based split for event/risk workflows.
- c. No train/test leakage.
- d. No entity leakage where applicable.
- e. Baseline model comparison.
- f. Class imbalance handling.
- g. Threshold analysis.
- h. Calibration analysis.
- i. Subgroup/cohort analysis.
- j. Robustness tests.
- k. Drift analysis.
- l. Backtesting.
- m. Shadow-mode evaluation.
- n. Canary evaluation before enforcement.
- o. Human review path for high-impact decisions.
- p. Model card or release report.

13.B. Recommended metrics

Sl. #	Metric	Purpose
1	PR-AUC	Rare-event performance
2	ROC-AUC	Secondary separability metric
3	Precision at fixed alert volume	Review capacity control
4	Recall at fixed FPR	Detection at tolerated friction
5	False-positive rate	User/customer impact
6	False-negative rate	Missed-risk impact

Sl. #	Metric	Purpose
7	Calibration error	Score trustworthiness
8	Brier score	Probability quality
9	Detection latency	Time to alert
10	Alert volume per 1,000 entities	Operational burden
11	Cost-weighted utility	Business-risk tradeoff
12	Lift over baseline	Practical improvement
13	Top-K precision	Analyst queue quality
14	Prediction latency	Runtime performance
15	Missing feature rate	Data quality
16	Drift score	Distribution stability

13.C. Required tests

- a. Held-out evaluation.
- b. Time-split evaluation.
- c. Baseline comparison.
- d. Rare-class metric threshold.
- e. Calibration test.
- f. Threshold test.
- g. Cohort regression test.
- h. Reproducibility test with fixed seed.
- i. Model artifact metadata test.
- j. Model loading test.
- k. Scoring timeout test.
- l. Invalid input test.
- m. NaN/Inf output rejection.
- n. Shadow-mode report required.
- o. Canary acceptance gates.

13.D. Stop-ship examples

- a. Model evaluated on training data.
- b. Accuracy-only release metric.
- c. No threshold selected.
- d. No calibration.
- e. No cold-start path.
- f. No rollback model.
- g. No model version in prediction logs.
- h. No human-readable release report for high-impact predictions.

13.E. Runbook TODO

- a. Model degradation playbook.
- b. Threshold rollback guide.
- c. Shadow-mode review process.
- d. Canary model launch process.
- e. Drift triage process.

13.F. Notes

- a. Do not use training accuracy as a production release metric.
- b. Rare-event workflows require imbalance-aware metrics.

14. MODEL SERVING & ONLINE SCORING

14.A. Required controls

- a. Model registry.
- b. Signed model artifacts.
- c. Feature schema compatibility check.
- d. Model version in every prediction.
- e. Feature version in every prediction.
- f. Decision threshold version in every prediction.
- g. Timeout around scoring.
- h. Fallback path.
- i. Circuit breaker around scorer.
- j. Canary routing.
- k. Shadow mode.
- l. Rollback to previous model.
- m. Safe default behavior.
- n. Prediction audit logs.
- o. Prediction metrics.
- p. Batch & online parity checks.

14.B. Required prediction metadata

Sl. #	Field	Purpose
1	prediction_id	Prediction traceability
2	model_version	Model audit
3	feature_schema_version	Feature audit
4	threshold_version	Decision audit
5	score	Raw score
6	calibrated_score	Probability-like score
7	decision	Outcome
8	reason_codes	Explainability where appropriate
9	score_time	Temporal audit
10	trace_id	Distributed trace
11	fallback_used	Reliability signal
12	feature_missing_count	Data quality
13	latency_ms	Performance

14.C. Required tests

- a. Unknown entity.
- b. Empty history.
- c. Missing feature.
- d. Schema mismatch.

- e. Model artifact missing.
- f. Model load failure.
- g. Prediction timeout.
- h. NaN/Inf score.
- i. Score outside expected range.
- j. Canary route.
- k. Rollback route.
- l. Shadow-mode non-enforcement.
- m. Audit log emitted.
- n. Sensitive data not logged.

14.D. Stop-ship examples

- a. Model scoring blocks critical path without timeout.
- b. Missing features cause crash.
- c. No fallback behavior.
- d. Model version absent from prediction logs.
- e. Rollback model unavailable.
- f. Shadow-mode not supported.

14.E. Runbook TODO

- a. Model rollback command.
- b. Threshold rollback command.
- c. Disable enforcement command.
- d. Scorer outage procedure.
- e. Feature-store outage procedure.

14.F. Notes

- a. Scoring should fail safe according to domain risk policy.

15. SCALABILITY & PERFORMANCE

15.A. Required controls

- a. Bounded memory use.
- b. Bounded concurrency.
- c. Bounded queue depth.
- d. Backpressure.
- e. Streaming/chunked processing.
- f. Incremental projection updates.
- g. Incremental feature updates.
- h. Batch/coalesce small operations where safe.
- i. Partitioned training.
- j. Partitioned replay.
- k. Hot partition detection.
- l. Horizontal workers.
- m. Work stealing where safe.
- n. Serverless/lambda burst controls.
- o. Connection pooling.
- p. Keep-alives.
- q. HTTP/2 or HTTP/3 where beneficial.
- r. Producer backpressure.
- s. Load shedding where needed.

- t. Resource limits in containers/lambda.

15.B. Performance metrics

Sl. #	Metric	Purpose
1	Throughput	Work completed
2	p50 latency	Typical latency
3	p95 latency	Tail trend
4	p99 latency	Tail SLO
5	Queue depth	Backlog
6	Queue age	User-impact risk
7	Consumer lag	Ingestion delay
8	Replay duration	Recovery time
9	Projection write latency	Store health
10	Storage conflict rate	Concurrency pressure
11	Retry rate	Dependency health
12	DLQ rate	Data quality
13	Memory per event	Efficiency
14	CPU per event	Efficiency
15	File descriptors	Resource risk
16	Network bytes	Transfer cost
17	Model scoring latency	ML serving health
18	Feature build latency	ML pipeline health
19	Training runtime	ML ops cost
20	Cost per 1,000 events	Unit economics
21	Cost per prediction	Unit economics

15.C. Required tests

- a. Load test.
- b. Soak test.
- c. Spike test.
- d. Hot-key test.
- e. Large-history account/entity test.
- f. Replay benchmark.
- g. Training benchmark.
- h. Scoring latency benchmark.
- i. Memory ceiling test.
- j. Connection pool saturation test.
- k. Queue backpressure test.
- l. Lambda cold-start test.
- m. Scale-out test.
- n. Scale-in graceful drain test.

15.D. Stop-ship examples

- a. Full history loaded into memory without bounds.
- b. Unbounded task/goroutine/future creation.
- c. No backpressure.
- d. No hot-key strategy.
- e. No p99 latency test.
- f. No memory limit test.
- g. No cost estimate.

15.E. Runbook TODO

- a. Scaling guide.
- b. Hot partition guide.
- c. Backpressure procedure.
- d. Capacity expansion guide.
- e. Cost anomaly investigation guide.

15.F. Notes

- a. Long-history event streams require replay & feature computation strategies that do not assume memory is unlimited.

16. HORIZONTAL SCALING, LAMBDA & SANDBOXING

16.A. Required controls

- a. Stateless workers where possible.
- b. Partition-aware worker assignment.
- c. Per-tenant isolation.
- d. Per-workload isolation.
- e. Lambda/serverless support for bursty ingestion where appropriate.
- f. Queue leases.
- g. Visibility timeouts.
- h. Heartbeats.
- i. Idempotent completion writes.
- j. Safe resume after crash.
- k. Graceful scale-in.
- l. Work stealing with ordering safeguards.
- m. Container or microVM isolation for risky ingestion.
- n. Rust/Go hardened parsing runtimes for untrusted payload validation where appropriate.
- o. CPU, memory, network, process, & filesystem limits.
- p. Restricted egress.
- q. Read-only filesystem where possible.
- r. No ambient credentials.
- s. Short-lived scoped tokens.
- t. Runtime provenance logging.

16.B. Sandbox metrics

Sl. #	Metric	Purpose
1	container_start_latency	Startup overhead
2	lambda_cold_start_latency	Serverless overhead

Sl. #	Metric	Purpose
3	sandbox_crash_count	Runtime stability
4	sandbox_timeout_count	Execution health
5	memory_limit_breach_count	Resource safety
6	cpu_throttle_count	Resource saturation
7	network_egress_violation_count	Security control
8	parser_failure_rate	Input quality
9	quarantine_rate	Risk rate
10	image_digest	Provenance
11	runtime_version	Runtime audit
12	policy_version	Sandbox policy
13	input_hash	Input traceability
14	output_hash	Output traceability

16.C. Required tests

- a. Worker crash mid-event.
- b. Lambda termination mid-job.
- c. Visibility timeout redelivery.
- d. Lease loss.
- e. Heartbeat failure.
- f. Container resource limit.
- g. Restricted egress.
- h. Malformed payload crash isolation.
- i. Sandbox provenance emitted.
- j. Runtime image scan.
- k. Multi-arch image test.

16.D. Stop-ship examples

- a. Ingestion can execute untrusted parsing in shared privileged runtime.
- b. Container has broad credentials.
- c. No resource limits.
- d. No restricted egress.
- e. Lambda retry is not idempotent.
- f. No visibility timeout handling.

16.E. Runbook TODO

- a. Sandbox crash triage.
- b. Quarantine review process.
- c. Container image rollback.
- d. Lambda retry storm mitigation.
- e. Runtime provenance lookup.

16.F. Notes

- a. Sandboxing reduces blast radius for malformed, malicious, or high-risk ingestion payloads.

17. OBSERVABILITY, LOGGING, METRICS & ALERTING

17.A. Required identifiers

- a. request_id
- b. trace_id
- c. correlation_id
- d. event_id
- e. job_id
- f. aggregate_id or hashed entity ID
- g. tenant_id
- h. partition_id
- i. consumer_group
- j. worker_id
- k. container_id
- l. lambda_invocation_id
- m. runtime_id
- n. model_version
- o. feature_schema_version
- p. deployment_version
- q. canary_version

17.B. Required metrics

Sl. #	Metric group	Metrics
1	Ingestion	events_received, events_processed, events_failed, events_retried, events_deduped
2	DLQ/quarantine	events_quarantined, dlq_count
3	Ack	ack_success_count, ack_failure_count
4	Latency	processing_latency, consumer_lag, queue_depth, queue_age
5	Projection	projection_conflict_count, projection_write_latency
6	Validation	schema_validation_failure_count, unknown_event_type_count
7	Versioning	version_gap_count, stale_event_count, duplicate_event_count
8	HTTP	request_count, error_count, latency_p50/p95/p99, retry_count, timeout_count
9	Rate limit	rate_limit_remaining, rate_limit_reset, retry_after_seconds, throttled_request_count
10	ML	prediction_count, prediction_latency, score_distribution, missing_feature_rate, drift

17.C. Severity alerts

Sl. #	Severity	Alert examples
1	Sev-1	Event loss, ack-after-failure spike, projection corruption, plaintext credential detection, secret/PII leak, DLQ explosion

Sl. #	Severity	Alert examples
2	Sev-2	Sustained processing lag, high retry budget burn, elevated 5xx/storage failures, high validation failure rate, circuit breaker open
3	Sev-2.5	p99 latency regression, hot partition, missing feature spike, sandbox crash loop, lambda cold-start spike
4	Sev-3	Non-critical warning increase, dashboard gap, documentation drift

17.D. Required tests

- a. Metrics emitted on success.
- b. Metrics emitted on failure.
- c. Alerts fire for Sev-1 simulation.
- d. Logs include correlation IDs.
- e. Logs redact secrets & PII.
- f. Trace spans propagate across queue, worker, storage, & scorer.
- g. Dashboard panels validate against metric schema.

17.E. Stop-ship examples

- a. No correlation IDs.
- b. No DLQ metrics.
- c. No projection drift metric.
- d. No model drift metric.
- e. Alerts do not distinguish severity.
- f. Logs include secrets or PII.

17.F. Runbook TODO

- a. Alert response guide.
- b. Dashboard ownership.
- c. On-call routing.
- d. Incident timeline reconstruction guide.
- e. SLO breach review template.

17.G. Notes

- a. Observability is part of correctness for event-sourced & ML systems.

18. TESTING MATRIX

18.A. Unit tests

- a. Event handlers.
- b. Version transitions.
- c. Duplicate handling.
- d. Payload validation.
- e. Retry classification.
- f. Rate-limit header parsing.
- g. Backoff/jitter math.
- h. Token comparison.
- i. Password hashing.
- j. Feature extraction.

- k. Feature vector ordering.
- l. Model scoring wrapper.
- m. Cold-start scoring.
- n. Error classification.

18.B. Validation tests

- a. Malformed JSON.
- b. Malformed NDJSON.
- c. Malformed XML.
- d. Malformed CSV.
- e. Malformed HTML.
- f. Invalid multipart.
- g. Unknown content type.
- h. Missing required field.
- i. Wrong field type.
- j. Oversized payload.
- k. Invalid timestamp.
- l. Unknown event type.
- m. Unsupported schema version.
- n. Secret/PII in forbidden fields.
- o. NaN/Inf model input.
- p. Invalid model output.

18.C. Regression tests

- a. Known event histories replay to expected state.
- b. Duplicate redelivery does not change state twice.
- c. Out-of-order event rejected/delayed/quarantined.
- d. Historical schema remains replayable.
- e. Previous incidents remain fixed.
- f. Model metrics do not regress beyond tolerance.
- g. Canary/control deltas remain within threshold.
- h. Feature schema remains compatible.

18.D. Integration tests

- a. Broker to consumer to projection store.
- b. Ack only after durable save.
- c. Save failure triggers retry/no ack.
- d. DLQ/quarantine path.
- e. Optimistic concurrency conflict.
- f. Projection rebuild.
- g. HTTP dependency with 429/Retry-After.
- h. HTTP dependency with 5xx.
- i. Feature store read/write.
- j. Model registry load.
- k. Scoring service prediction.
- l. Observability emission.
- m. Alert routing.

18.E. Contract tests

- a. Event schema.
- b. API schema.
- c. Storage interface.
- d. Broker interface.

- e. Model artifact schema.
- f. Feature schema.
- g. Vendor SDK behavior.
- h. HTTP response & error body contracts.

18.F. Fault-injection tests

- a. Store timeout.
- b. Store partial write.
- c. Broker redelivery.
- d. Ack failure.
- e. Network timeout.
- f. DNS failure.
- g. TLS failure.
- h. 429 with `Retry-After`.
- i. 503 with `Retry-After`.
- j. 500/502/504.
- k. Consumer crash.
- l. Lambda termination.
- m. Container kill.
- n. Corrupt payload.
- o. Duplicate storm.
- p. Hot partition.
- q. Circuit breaker open.
- r. Clock skew.
- s. Disk full.
- t. Memory pressure.
- u. File descriptor exhaustion.

18.G. Security tests

- a. Secret scanning.
- b. PII redaction.
- c. Authn/authz.
- d. Token expiry.
- e. Token single-use.
- f. Constant-time comparison policy.
- g. Injection tests.
- h. SSRF tests.
- i. Path traversal tests.
- j. Dependency scan.
- k. Container scan.
- l. IaC scan.
- m. Least privilege test.
- n. Egress restriction test.

18.H. ML tests

- a. Time-based split.
- b. Leakage test.
- c. Feature/label window alignment.
- d. Baseline comparison.
- e. PR-AUC threshold.
- f. Recall/FPR threshold.
- g. Calibration.
- h. Cohort performance.
- i. Drift detection.

- j. Reproducibility.
- k. Feature schema compatibility.
- l. Offline/online parity.
- m. Shadow-mode evaluation.
- n. Canary evaluation.
- o. Rollback model load.

18.I. Performance tests

- a. Load.
- b. Spike.
- c. Soak.
- d. Replay benchmark.
- e. Training benchmark.
- f. Scoring latency.
- g. Memory ceiling.
- h. Queue backpressure.
- i. Connection pool saturation.
- j. Hot-key load.
- k. Multi-tenant fairness.
- l. Cost benchmark.

18.J. Notes

- a. Tests should cover success, failure, retry, rollback, & observability paths.
- b. Synthetic tests should include large histories & duplicate storms.
- c. AI-generated code should have stricter tests around hidden assumptions.

19. CI/CD QUALITY GATES

19.A. Required pre-merge gates

- a. Formatting.
- b. Linting.
- c. Static analysis.
- d. Type checking.
- e. Unit tests.
- f. Validation tests.
- g. Regression tests.
- h. Integration tests.
- i. Security tests.
- j. Secret scan.
- k. Dependency vulnerability scan.
- l. License scan.
- m. SBOM generation.
- n. Container image scan.
- o. Infrastructure-as-code scan.
- p. Coverage threshold.
- q. Changed-line coverage threshold.
- r. Build reproducibility check.
- s. Artifact signing.
- t. Test artifact publishing.

19.B. Required coverage

Sl. #	Coverage gate	Requirement
1	Overall line coverage	$\geq 80-85\%$
2	Changed-line coverage	$\geq$ agreed threshold
3	Unit coverage	$\geq w\%$
4	Validation coverage	$\geq x\%$
5	Regression coverage	$\geq y\%$
6	Integration coverage	$\geq z\%$
7	Critical-path coverage	Higher than general coverage
8	Security-sensitive code	Explicit direct tests required

19.C. Required release gates

- a. All tests pass.
- b. No Sev-1 findings.
- c. Sev-2 findings mitigated or explicitly approved.
- d. SBOM attached.
- e. License report attached.
- f. CVE report attached.
- g. Third-party notices generated.
- h. Model card or release report attached where applicable.
- i. Canary plan attached.
- j. Rollback plan attached.
- k. Runbook attached.
- l. SLO dashboard exists.
- m. Alert routing tested.
- n. Data migration/replay plan tested.
- o. Versioned artifact created.
- p. Deployment diff reviewed.

19.D. Stop-ship examples

- a. Critical/high CVE unpatched without waiver.
- b. Unknown or incompatible license.
- c. No SBOM.
- d. Secret scan failure.
- e. Coverage below gate.
- f. Flaky critical tests.
- g. No rollback plan.
- h. No canary plan.
- i. No production runbook.
- j. No model validation report.
- k. No event replay test.

19.E. Runbook TODO

- a. CI failure triage guide.
- b. Emergency waiver process.
- c. Artifact promotion process.
- d. Release approval checklist.
- e. Rollback validation checklist.

19.F. Notes

- a. Coverage thresholds should not replace critical-path test design.

20. DEPLOYMENT, CANARY, FEATURE FLAGS & ROLLBACK

20.A. Required deployment controls

- a. Immutable versioned artifacts.
- b. Blue/green or rolling deployment.
- c. Canary deployment.
- d. Shadow mode for ML.
- e. Feature flags.
- f. Kill switches.
- g. Config versioning.
- h. Safe default config.
- i. Gradual rollout by tenant, region, tier, traffic percentage, event type, or model use case.
- j. Automatic rollback triggers.
- k. Manual rollback runbook.
- l. Database/schema compatibility.
- m. Event schema compatibility.
- n. Model schema compatibility.
- o. Backward-compatible config.
- p. No irreversible migration without tested rollback or forward-fix plan.

20.B. Canary stages

Sl. #	Stage	Rollout
1	a	0% enforcement, shadow mode only
2	b	Internal tenants
3	c	Low-risk tenant cohort
4	d	1% traffic
5	e	5% traffic
6	f	10% traffic
7	g	25% traffic
8	h	50% traffic
9	i	100% rollout after SLO stability

20.C. Canary metrics

- a. Error rate.
- b. p95/p99 latency.
- c. Queue lag.
- d. DLQ count.
- e. Retry rate.
- f. Duplicate rate.
- g. Projection conflict rate.
- h. Memory/CPU.
- i. Cost per event.

- j. Prediction latency.
- k. Score distribution.
- l. Threshold crossing rate.
- m. Missing feature rate.
- n. Fallback rate.
- o. Alert volume.
- p. User-impact rate.
- q. Canary/control metric delta.
- r. Security events.
- s. Business KPIs where applicable.

20.D. Rollback triggers

- a. Sev-1 alert.
- b. Error rate above threshold.
- c. p99 latency above threshold.
- d. DLQ spike.
- e. Projection drift detected.
- f. Credential/PII leak.
- g. Critical dependency issue.
- h. Model score distribution shift.
- i. False-positive spike.
- j. Missing feature spike.
- k. Canary worse than control beyond threshold.
- l. Cost spike.
- m. Manual incident commander decision.

20.E. Rollback requirements

- a. Previous artifact available.
- b. Previous model available.
- c. Previous config available.
- d. Previous feature flag state available.
- e. Database migration compatibility understood.
- f. Event schema compatibility understood.
- g. Projection rebuild path available.
- h. DLQ replay plan available.
- i. Customer communication plan where needed.
- j. Incident ticket generated.
- k. Post-rollback verification checklist.

20.F. Stop-ship examples

- a. No rollback artifact.
- b. No canary metrics.
- c. No kill switch.
- d. No shadow mode for enforcement model.
- e. Migration cannot roll back or forward-fix.
- f. New events not readable by previous consumer.
- g. New model feature schema not compatible with fallback model.

20.G. Runbook TODO

- a. Rollback command.
- b. Roll-forward command.
- c. Canary pause command.
- d. Canary resume command.

- e. Feature-flag disable command.
- f. Model enforcement disable command.
- g. DLQ replay after rollback.

20.H. Notes

- a. ML enforcement should move through shadow mode before canary.

21. SYSTEM PACKAGE, DEPENDENCY, VENDOR & LICENSE GOVERNANCE

21.A. Required controls

- a. Pin all language dependencies.
- b. Pin all system packages.
- c. Pin base images by digest.
- d. Pin language runtimes.
- e. Pin ML framework versions.
- f. Pin vendor SDK versions.
- g. Use lockfiles.
- h. Generate SBOMs.
- i. Run CVE scans.
- j. Run license scans.
- k. Run secret scans.
- l. Run container scans.
- m. Track transitive dependencies.
- n. Track end-of-life dates.
- o. Track vendor deprecations.
- p. Track migration paths.
- q. Track replacement candidates.
- r. Track commercial-use rights.
- s. Track resale/productization rights.
- t. Generate third-party notices.
- u. Review high-risk licenses.
- v. Review model & dataset licenses.
- w. Review generated artifact rights where applicable.

21.B. Vendor maintenance criteria

Sl. #	Criteria	Why it matters
1	Active release cadence	Reduces stale dependency risk
2	Responsive CVE handling	Reduces security exposure
3	Public security advisories	Supports vulnerability monitoring
4	Clear deprecation policy	Prevents surprise breakage
5	Stable APIs	Reduces migration churn
6	Migration documentation	Lowers replacement cost
7	Commercial support availability	Supports enterprise ops
8	Clear licensing	Supports productization
9	Predictable pricing	Supports cost planning

Sl. #	Criteria	Why it matters
10	Low lock-in	Supports migration flexibility

21.C. License checks

- a. Commercial use permitted.
- b. SaaS use permitted.
- c. Resale permitted.
- d. Redistribution permitted.
- e. Modification permitted.
- f. Static/dynamic linking obligations understood.
- g. Container redistribution permitted.
- h. Model artifact distribution permitted.
- i. Dataset use permitted.
- j. Attribution obligations tracked.
- k. Source disclosure obligations reviewed.
- l. Patent terms reviewed.
- m. Field-of-use restrictions reviewed.
- n. Non-commercial/research-only restrictions blocked unless approved.
- o. Unknown license blocked.

21.D. Required metrics

- a. Current supported dependency percentage.
- b. Critical/high/medium/low CVE count.
- c. Mean time to patch CVEs.
- d. EOL package count.
- e. Unknown license count.
- f. Incompatible license count.
- g. Dependencies requiring legal review.
- h. SBOM coverage percentage.
- i. Signed artifact percentage.
- j. Unpinned dependency count.
- k. Dependency update backlog.
- l. Vendor maintenance score.
- m. Migration cost estimate.
- n. Replacement candidate coverage.
- o. Third-party notice completeness.
- p. Commercial-use compliance status.

21.E. Required gates

- a. Critical CVE blocks release unless patched, mitigated, or formally waived.
- b. High CVE blocks production unless risk accepted with compensating control.
- c. Unknown license blocks release.
- d. Incompatible license blocks release.
- e. Unsupported runtime blocks release.
- f. EOL base image blocks release.
- g. Missing SBOM blocks release.
- h. Missing license report blocks release.
- i. Missing third-party notices blocks release where redistribution applies.
- j. ML model/dataset commercial-use uncertainty blocks productization.

21.F. Stop-ship examples

- a. Production image uses end-of-life OS base image.

- b. Critical/high CVE remains unpatched without mitigation or waiver.
- c. Dependency has incompatible license for commercial resale or productization.
- d. Package license is unknown, missing, or ambiguous.
- e. Critical dependency is unmaintained or abandoned.
- f. Runtime version is unsupported by vendor.
- g. Container image lacks SBOM or scan results.
- h. Model, dataset, or pretrained asset has research-only or non-commercial terms.
- i. Build pulls unpinned latest packages at release time.
- j. Transitive dependency uses prohibited license.

21.G. Runbook TODO

- a. CVE patch procedure.
- b. License escalation procedure.
- c. Vendor deprecation migration process.
- d. Dependency rollback guide.
- e. SBOM generation instructions.
- f. Third-party notices generation instructions.

21.H. Notes

- a. Dependency choice is a long-term operational & legal commitment.
- b. Cost-effectiveness should include migration, support, security maintenance, & scaling cost.

22. CONFIGURATION, POLICY & MIGRATION

22.A. Required controls

- a. Typed configuration.
- b. Config validation at startup.
- c. Safe defaults.
- d. Environment overlays.
- e. Secret references, not secret values.
- f. Hot reload where safe.
- g. Config versioning.
- h. Policy-as-code for quotas, routing, admission, rollout, & enforcement.
- i. Feature flags.
- j. Migration scripts.
- k. Migration tests.
- l. Backward compatibility.
- m. Forward compatibility.
- n. Rollback strategy.
- o. Drift detection between desired & actual config.
- p. Audit log for config changes.

22.B. Required questions

- a. Does config fail closed?
- b. Are limits explicit?
- c. Are defaults safe?
- d. Can config change be audited?
- e. Can config be rolled back?
- f. Are policy decisions logged?
- g. Can old workers read new config?
- h. Can new workers read old config?

- i. Are migrations reversible or forward-fixable?
- j. Are event schemas compatible across deploy versions?

22.C. Required tests

- a. Invalid config fails startup.
- b. Missing config uses safe defaults or fails closed.
- c. Config hot reload.
- d. Config rollback.
- e. Policy rule test.
- f. Migration forward test.
- g. Migration rollback or forward-fix test.
- h. Mixed-version deployment test.

22.D. Stop-ship examples

- a. Unsafe default enables enforcement.
- b. Config contains raw secrets.
- c. Policy change has no audit log.
- d. Migration cannot be rolled back or forward-fixed.
- e. New schema breaks old workers.

22.E. Runbook TODO

- a. Config rollback procedure.
- b. Policy change approval process.
- c. Migration incident procedure.
- d. Mixed-version deploy procedure.

22.F. Notes

- a. Feature flags should default to safe behavior.

23. COST, CAPACITY & OPERATIONAL EFFICIENCY

23.A. Required cost controls

- a. Cost per event.
- b. Cost per prediction.
- c. Cost per tenant.
- d. Cost per retraining run.
- e. Storage growth forecast.
- f. Event log retention cost.
- g. Feature store cost.
- h. Model serving cost.
- i. DLQ/replay cost.
- j. Network egress cost.
- k. Vendor API cost.
- l. Lambda cold-start/cost tradeoff.
- m. Container idle cost.
- n. GPU/CPU training cost.
- o. Alert review cost.
- p. False-positive operational cost.
- q. False-negative business/security cost.

23.B. Required questions

- a. Is batching used where safe?
- b. Is streaming used for large payloads?
- c. Are duplicate operations deduped to reduce cost?
- d. Are hot tenants isolated?
- e. Are vendor API calls cached where allowed?
- f. Is model complexity justified by lift?
- g. Is retraining frequency justified?
- h. Is storage growth sustainable?
- i. Is horizontal scaling cost bounded?
- j. Are high-cardinality metrics controlled?

23.C. Required tests

- a. Cost benchmark.
- b. Load-cost projection.
- c. Duplicate storm cost.
- d. Replay cost estimate.
- e. Training cost estimate.
- f. Scoring cost estimate.
- g. Tenant fairness cost test.
- h. Budget alert test.

23.D. Stop-ship examples

- a. No cost estimate for replay.
- b. No cost estimate for model training.
- c. High-cardinality metrics can explode bill.
- d. Duplicate storm multiplies vendor charges.
- e. Model complexity has no lift justification.
- f. Storage growth forecast absent.

23.E. Runbook TODO

- a. Cost anomaly triage.
- b. Budget alert procedure.
- c. Expensive tenant isolation.
- d. Replay cost approval process.

23.F. Notes

- a. Cost is part of production safety when replay, training, or retry storms can scale unexpectedly.

24. DOCUMENTATION & RUNBOOKS

24.A. Required documentation

- a. System overview.
- b. Event schema catalog.
- c. Projection semantics.
- d. Idempotency strategy.
- e. Ordering strategy.
- f. Retry/backoff policy.
- g. Rate-limit header handling policy.

- h. Timeout policy.
- i. DLQ/quarantine policy.
- j. Security model.
- k. Data classification.
- l. PII/secret redaction policy.
- m. Model card.
- n. Feature schema.
- o. Model validation report.
- p. Deployment plan.
- q. Canary plan.
- r. Rollback plan.
- s. Incident runbook.
- t. On-call playbook.
- u. SLOs.
- v. Alert definitions.
- w. Dependency/license policy.
- x. Migration guide.
- y. Local dev guide.
- z. Replay/reconciliation guide.

24.B. Required runbook actions

- a. How to pause consumers.
- b. How to drain queues.
- c. How to inspect DLQ.
- d. How to replay safely.
- e. How to rebuild projections.
- f. How to roll back deployment.
- g. How to roll back model.
- h. How to disable enforcement.
- i. How to rotate secrets.
- j. How to handle credential/PII leak.
- k. How to handle projection drift.
- l. How to handle rate-limit outage.
- m. How to handle vendor dependency outage.
- n. How to validate recovery.
- o. Who owns each escalation path.

24.C. Stop-ship examples

- a. No runbook.
- b. No rollback instructions.
- c. No DLQ replay instructions.
- d. No model rollback instructions.
- e. No incident severity definitions.
- f. No ownership for critical alerts.
- g. No dependency/license policy.
- h. No event replay guide.

24.D. Final acceptance checklist

24.D.a. Event ingestion

- ☐ Event schemas versioned.
- ☐ Event IDs stable.
- ☐ Aggregate versions monotonic.

- ☐ Per-aggregate ordering enforced or validated.
- ☐ Duplicate delivery idempotent.
- ☐ Ack only after durable success.
- ☐ Invalid events rejected or quarantined.
- ☐ Unknown event types do not advance state.
- ☐ Projection writes use optimistic concurrency.
- ☐ Projection replay deterministic.
- ☐ Projection rebuild test passes.
- ☐ DLQ/quarantine implemented.
- ☐ Retry/backoff/jitter implemented.
- ☐ Rate-limit headers honored.
- ☐ Dynamic throttling implemented.
- ☐ Timeouts implemented.
- ☐ Circuit breakers implemented.
- ☐ Backpressure implemented.
- ☐ Concurrency bounded.
- ☐ Structured logs emitted.
- ☐ Metrics emitted.
- ☐ Sev alerts configured.

24.D.b. Security & compliance

- ☐ No plaintext secrets.
- ☐ No secrets in immutable logs.
- ☐ No sensitive PII in logs.
- ☐ Tokens hashed/scoped/expiring/single-use where applicable.
- ☐ Authn/authz enforced.
- ☐ TLS enforced.
- ☐ Secret scan passes.
- ☐ SAST/DAST/IaC scans pass as applicable.
- ☐ CVE scan passes.
- ☐ License scan passes.
- ☐ SBOM generated.
- ☐ Third-party notices generated.
- ☐ Commercial/resale/productization rights validated.
- ☐ Data retention & access controls defined.

24.D.c. ML/risk model

- ☐ Point-in-time features.
- ☐ No future leakage.
- ☐ Feature schema explicit.
- ☐ Offline/online parity tested.
- ☐ Cold-start path.
- ☐ Missing feature path.
- ☐ Held-out time-split validation.
- ☐ Accuracy not used alone.
- ☐ PR-AUC/precision/recall/FPR/calibration evaluated.
- ☐ Threshold selected by cost & operational capacity.
- ☐ Cohort metrics reviewed.
- ☐ Model artifact versioned.
- ☐ Model card/release report created.
- ☐ Shadow mode completed.
- ☐ Canary completed.

- ☐ Rollback model available.
- ☐ Drift monitoring enabled.

24.D.d. CI/CD

- ☐ Formatting passes.
- ☐ Linting passes.
- ☐ Static analysis passes.
- ☐ Type checking passes.
- ☐ Unit tests pass.
- ☐ Validation tests pass.
- ☐ Regression tests pass.
- ☐ Integration tests pass.
- ☐ Security tests pass.
- ☐ Performance tests pass where required.
- ☐ Fault-injection tests pass.
- ☐ Coverage ≥ 80 -85% overall.
- ☐ Per-suite coverage thresholds met.
- ☐ Changed-line coverage gate met.
- ☐ Build reproducible.
- ☐ Artifact signed.
- ☐ Container image scanned.
- ☐ Deployment manifest reviewed.
- ☐ Canary plan attached.
- ☐ Rollback plan attached.
- ☐ Runbook attached.

24.D.e. Deployment

- ☐ Feature flags configured.
- ☐ Kill switch configured.
- ☐ Canary metrics defined.
- ☐ Rollback triggers defined.
- ☐ Previous artifact available.
- ☐ Previous config available.
- ☐ Previous model available.
- ☐ Mixed-version compatibility tested.
- ☐ Migration tested.
- ☐ Alert routing tested.
- ☐ Dashboard ready.
- ☐ On-call owner assigned.

24.E. Notes

- a. Ship only when every Sev-1 is resolved.
- b. Ship only when Sev-2 risks are resolved or explicitly mitigated with owner, expiry, & rollback.
- c. Use canary before broad rollout.
- d. Use shadow mode before ML enforcement.
- e. Keep rollback fast, tested, & documented.
- f. Treat ingestion correctness, credential safety, model validity, observability, supply-chain integrity, & license compliance as non-negotiable production gates.